

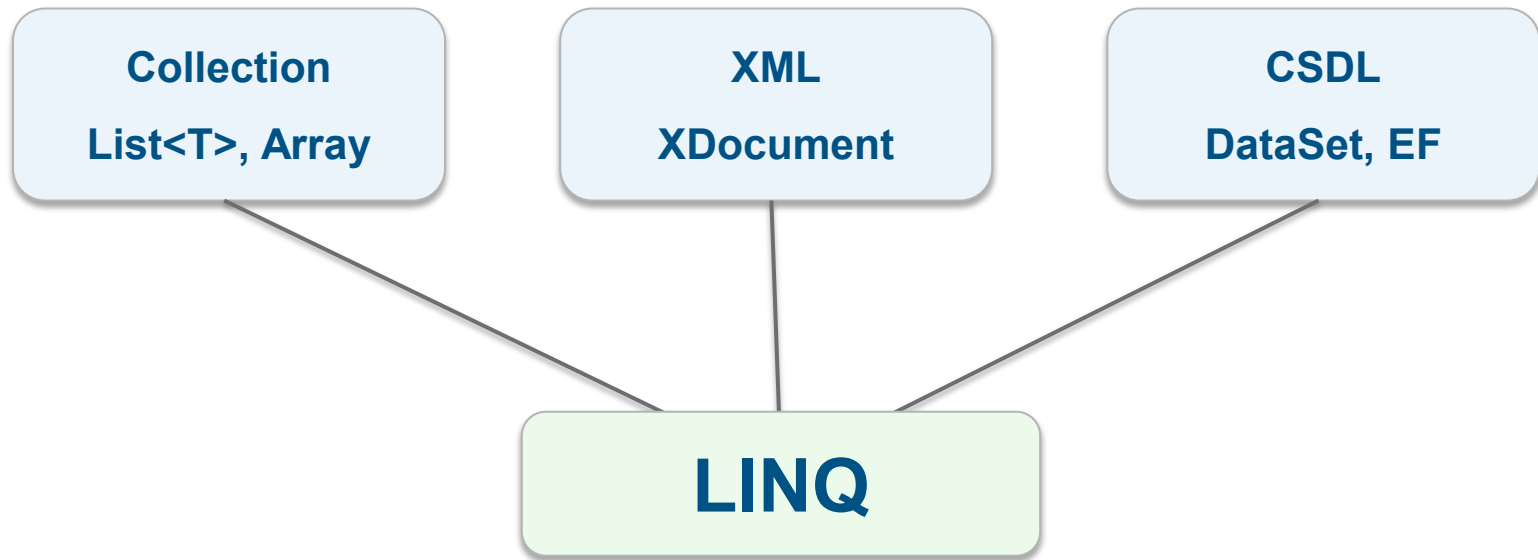
LINQ CƠ BẢN

Dữ liệu Collection, XML, CSDL: Một cú pháp truy vấn thống nhất

Mục tiêu bài học

- Hiểu LINQ là gì và vì sao giúp thống nhất cách truy vấn dữ liệu
- Phân biệt LINQ to Objects, LINQ to ADO.NET và LINQ to XML
- Viết được truy vấn với Query Expression syntax và Method syntax
- Biến đổi dữ liệu bằng select, kiểu vô danh, group và join
- Vận dụng LINQ trong bài tập quản lý danh sách đối tượng C#

Vấn đề: nhiều nguồn dữ liệu, nhiều cách truy cập



Một mô hình truy vấn trong C#
Ít phụ thuộc vào kiểu nguồn dữ liệu

1. Tổng quan về LINQ

- LINQ = Language Integrated Query: truy vấn được tích hợp trong ngôn ngữ C#
- Làm việc trực tiếp với object, collection, XML, DataSet, Entity...
- Truy vấn là biểu thức C#: có kiểm tra kiểu khi biên dịch, hỗ trợ IntelliSense
- Kết quả truy vấn thường là IEnumerable<T> hoặc IQueryable<T>

```
int[] numbers = { 0, 1, 2, 3, 4, 5, 6 };
```

```
var query = from n in numbers  
            where n % 2 == 0  
            orderby n descending  
            select n;
```

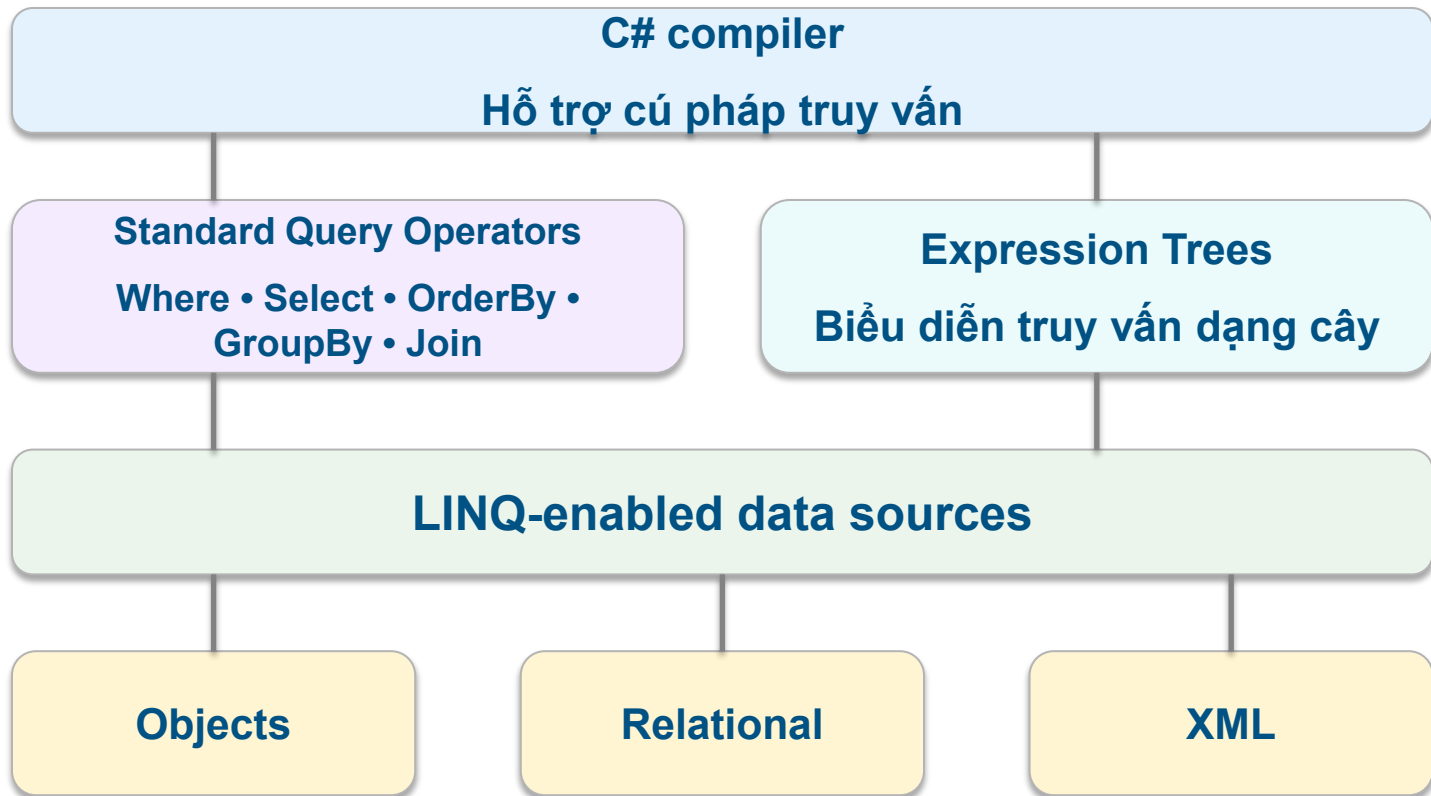
Namespace hỗ trợ LINQ

- System.Linq: các toán tử truy vấn chuẩn cho object/collection
- System.Data.Linq: LINQ to SQL (mô hình cũ, cần biết về lịch sử)
- System.Data.Entity hoặc Microsoft.EntityFrameworkCore: truy vấn qua Entity Framework
- System.Xml.Linq: LINQ to XML với XDocument, XElement

```
using System;  
using System.Collections.Generic;  
using System.Linq;
```

```
List<int> ds = new() { 5, 2, 9, 1 };  
var kq = ds.Where(x => x > 3).OrderBy(x => x);
```

Kiến trúc LINQ trong C#



Hoạt động của một truy vấn LINQ

Bước 1: nhận nguồn dữ liệu

```
// 1. Nguồn dữ liệu
```

```
int[] mangSo = { 50, 42, 16, 3, 9, 8, 12,  
7, 24, 0 };
```

Bước 2: tạo truy vấn

Bước 3: thi hành truy vấn

```
// 2. Tạo truy vấn
```

```
var query = from n in mangSo  
             where n <= 3  
             select n;
```

Truy vấn thường chưa chạy ngay khi khai báo

```
// 3. Thi hành truy vấn
```

```
foreach (int n in query)  
    Console.Write(n + " ");
```

Ghi nhớ: foreach / ToList() / Count() mới kích hoạt lấy dữ liệu

Deferred Execution: truy vấn chạy khi cần

- LINQ lưu “cách truy vấn”, không lưu ngay kết quả
- Dữ liệu thay đổi trước khi duyệt có thể làm kết quả thay đổi
- Dùng ToList(), ToArray() nếu muốn chốt kết quả tại thời điểm hiện tại

```
List<int> ds = new() { 1, 2, 3 };
```

```
var q = from x in ds  
        where x > 1  
        select x;           // chưa duyệt dữ liệu
```

```
ds.Add(4);
```

```
foreach (var x in q)  
    Console.Write(x + " "); // 2 3 4
```

q.ToList() = thực thi ngay và tạo danh sách mới

2. LINQ to Objects

- Dùng với các collection cài đặt IEnumerable hoặc IEnumerable<T>
- Làm việc tốt với List<T>, Array, Dictionary<TKey,TValue>, Queue, Stack...
- Dữ liệu thường nằm trong bộ nhớ của chương trình

```
List<string> monHoc = new()  
{  
    "Cơ sở dữ liệu", "Cấu trúc dữ liệu",  
    "Lập trình C#", "Trí tuệ nhân tạo"  
};
```

```
var q = from m in monHoc  
        where m.Contains("dữ liệu")  
        select m.ToUpper();
```

Hai cú pháp: Query Expression và Method Syntax

- Query Expression gần với SQL, dễ học lúc đầu
- Method Syntax dùng lambda, gọn và phổ biến trong dự án thực tế
- Trình biên dịch C# chuyển Query Expression về Method Syntax

```
// Query Expression syntax
var q1 = from n in mangSo
        where n <= 3
        select n;
```

```
// Method syntax
var q2 = mangSo
        .Where(n => n <= 3)
        .Select(n => n);
```

Cú pháp cơ bản của Query Expression

```
from <biến> in <nguồn dữ liệu>  
[where <điều kiện>]  
[orderby <biểu thức> ascending|descending]  
select <biểu thức>;
```

Bắt đầu với from

Lọc bằng where

**Kết thúc bằng
select / group**

```
var q = from sv in dsSinhVien  
        where sv.Diem >= 5  
        orderby sv.HoTen  
        select sv;
```

Lọc, sắp xếp, chiếu dữ liệu

- where: lọc phần tử
- orderby: sắp xếp tăng/giảm
- select: chọn toàn bộ object, một thuộc tính hoặc object mới
- new { ... }: tạo kiểu vô danh cho kết quả trung gian

```
var ketQua = from sv in dsSinhVien
              where sv.Diem >= 8
              orderby sv.Diem descending, sv.HoTen ascending
              select new
              {
                  sv.MaSV,
                  sv.HoTen,
                  XepLoai = "Giỏi"
              };
```

Biến đổi dữ liệu: kiểu mới và kiểu vô danh

- Truy vấn có thể không làm đổi dữ liệu nguồn
- Output có thể cùng kiểu, khác kiểu, hoặc kiểu vô danh
- var rất hữu ích khi kết quả là kiểu vô danh

```
class SinhVien
{
    public string MaSV { get; set; } = "";
    public string HoTen { get; set; } = "";
    public double Diem { get; set; }
}
```

```
var q = from sv in dsSinhVien
        select new { sv.HoTen, KetQua = sv.Diem >= 5 };
```

Không nên trả kiểu vô danh ra ngoài phạm vi method

Một số toán tử LINQ thường dùng

- Where: lọc
- Select: chiếu/biến đổi
- OrderBy/ThenBy: sắp xếp
- GroupBy: nhóm
- Join: kết dữ liệu
- Count/Sum/Average/Max/Min: tổng hợp
- Any/All: kiểm tra điều kiện
- First/Single: lấy phần tử
- Take/Skip: phân trang
- Distinct: loại trùng

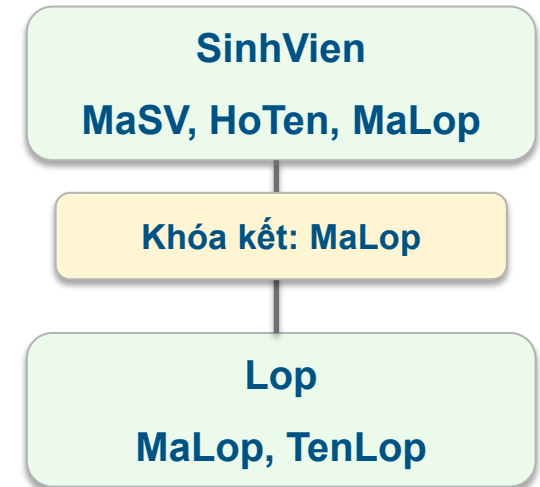
Nhóm dữ liệu với group by

- group ... by ... tạo các nhóm
- g.Key là khóa của nhóm
- Có thể tính Count, Average, Max, Min trên mỗi nhóm
- Dùng into để tiếp tục truy vấn sau khi nhóm

```
var nhomTheoLop = from sv in dsSinhVien
                  group sv by sv.Lop into g
                  select new
                  {
                      Lop = g.Key,
                      SoLuong = g.Count(),
                      DiemTB = g.Average(x => x.Diem)
                  };
```

Kết hợp dữ liệu với join

```
var q = from sv in dsSinhVien
        join lop in dsLop
        on sv.MaLop equals lop.MaLop
        select new
        {
            sv.HoTen,
            TenLop = lop.TenLop
        };
```



let và into: đặt tên kết quả trung gian

- let: khai báo biến trung gian trong truy vấn
- into: đặt tên cho kết quả group/join/select để truy vấn tiếp
- Giúp code dễ đọc khi điều kiện phức tạp

```
var q = from sv in dsSinhVien
        let dtb = sv.DiemLT * 0.4 + sv.DiemTH * 0.6
        where dtb >= 5
        orderby dtb descending
        select new { sv.HoTen, DiemTB = dtb };
```

```
var q2 = from sv in dsSinhVien
        group sv by sv.Lop into g
        where g.Count() >= 10
        select g;
```

Chuyển đổi kết quả truy vấn

- ToList / ToArray: tạo collection mới
- ToDictionary: tra cứu nhanh theo khóa
- Count / Average / Sum: trả về giá trị tổng hợp
- Chú ý dữ liệu rỗng khi dùng Average, First, Single

```
var q = dsSinhVien.Where(sv => sv.Diem >= 5);
```

```
List<SinhVien> dsDat = q.ToList();
```

```
SinhVien[] mangDat = q.ToArray();
```

```
Dictionary<string, SinhVien> traCuu =  
    dsSinhVien.ToDictionary(sv => sv.MaSV);
```

```
int soLuong = dsSinhVien.Count(sv => sv.Diem >= 5);
```

```
double diemTB = dsSinhVien.Average(sv => sv.Diem);
```

LINQ to XML: truy vấn tài liệu XML

- XDocument: đại diện tài liệu XML
- XElement: đại diện phần tử XML
- Descendants: lấy các node con theo tên
- Có thể truy vấn, tạo mới và cập nhật XML trong bộ nhớ

```
XDocument doc = XDocument.Load("monhoc.xml");
```

```
var q = from mh in doc.Descendants("MonHoc")  
        where (string)mh.Element("Loai") == "CNTT"  
        select new  
        {  
            Ma = (string)mh.Attribute("Ma"),  
            Ten = (string)mh.Element("Ten")  
        };
```

LINQ với dữ liệu quan hệ

- LINQ to DataSet: truy vấn dữ liệu dạng DataTable/DataSet
- LINQ to SQL: ánh xạ bảng CSDL sang object, hiện ít dùng trong dự án mới
- Entity Framework / EF Core: dùng IQueryable<T>, truy vấn được dịch thành SQL
- Cần phân biệt: LINQ to Objects chạy trong RAM, LINQ to Entities chạy ở CSDL khi còn là IQueryable

```
var q = db.SinhViens
    .Where(sv => sv.Diem >= 5)
    .OrderBy(sv => sv.HoTen)
    .Select(sv => new { sv.MaSV, sv.HoTen });
```

```
// SQL thường chỉ được gửi khi gọi ToList(), First(), Count()...
```

IEnumerable<T> và IQueryable<T>

- Phù hợp: List<T>, Array
- Where/Select là delegate chạy trong .NET
- Tải dữ liệu về rồi mới lọc nếu dùng sai thời điểm
- Phù hợp: EF Core DbSet<T>
- Biểu thức được nhà cung cấp dịch thành câu SQL
- Nên lọc trước khi ToList để tránh tải nhiều dữ liệu

IEnumerable<T>

Duyệt object trong bộ nhớ

IQueryable<T>

Biểu thức truy vấn có thể dịch sang SQL

Quy tắc thực hành: Filter → Project → Materialize

Ví dụ thực hành: quản lý sinh viên

- Yêu cầu sinh viên chạy từng phần
- Đổi sang Method Syntax
- Thêm cột Xếp loại
- Nhóm theo lớp và tính điểm trung bình

```
List<SinhVien> ds = new()  
{  
    new() { MaSV="SV01", HoTen="An", Lop="DCT124", Diem=8.2 },  
    new() { MaSV="SV02", HoTen="Bình", Lop="DCT124", Diem=4.5 },  
    new() { MaSV="SV03", HoTen="Chi", Lop="DCT123", Diem=7.0 },  
};  
  
var q = from sv in ds  
        where sv.Diem >= 5  
        orderby sv.Diem descending  
        select new { sv.MaSV, sv.HoTen, sv.Diem };
```

Bài tập 1: lọc và sắp xếp danh sách

- Tạo lớp SanPham gồm MaSP, TenSP, DonGia, SoLuong, Loai
- Tạo danh sách ít nhất 10 sản phẩm
- Dùng LINQ lấy sản phẩm có DonGia ≥ 100000 và còn hàng
- Sắp xếp giảm dần theo DonGia, sau đó tăng dần theo TenSP
- In ra MaSP, TenSP, DonGia, ThanhTien = DonGia * SoLuong

```
var q = from sp in dsSanPham
        where sp.DonGia >= 100000 && sp.SoLuong > 0
        orderby sp.DonGia descending, sp.TenSP
        select new { sp.MaSP, sp.TenSP, sp.DonGia, ThanhTien =
sp.DonGia * sp.SoLuong };
```

Bài tập 2: group và join

- Tạo 2 danh sách: Lop(MaLop, TenLop) và SinhVien(MaSV, HoTen, MaLop, Diem)
- Join để in HoTen, TenLop, Diem
- Group theo MaLop/TenLop để tính SoLuong và DiemTrungBinh
- Lọc các lớp có DiemTrungBinh ≥ 6.5
- Viết lại ít nhất một truy vấn bằng Method Syntax

Gợi ý: `join ... on ... equals ...; group ... by ... into g; g.Average(x => x.Diem)`

Lỗi thường gặp khi viết LINQ

- Quên using System.Linq nên không thấy Where, Select, OrderBy
- Nhầm “=” với “equals” trong join Query Expression
- Gọi First/Single trên tập rỗng làm phát sinh exception
- Gọi ToList quá sớm khi làm việc với CSDL khiến chương trình tải nhiều dữ liệu
- Dùng kiểu vô danh vượt ra ngoài phạm vi method
- Không kiểm tra null khi truy vấn object có thuộc tính có thể null

Tổng kết

- LINQ giúp truy vấn dữ liệu thống nhất ngay trong C#
- Ba bước chính: nguồn dữ liệu → tạo truy vấn → thi hành truy vấn
- Query Expression dễ đọc; Method Syntax phổ biến trong dự án thực tế
- select có thể trả object gốc, thuộc tính, class mới hoặc kiểu vô danh
- Nên lọc và chọn dữ liệu trước khi materialize bằng ToList/ToArray

Thông điệp chính: LINQ không chỉ để “lọc danh sách”, mà là tư duy xử lý dữ liệu trong C#.